Verifier-Gated LLM NetOps: A Preregistered, Artifact-Backed Evaluation of Input-Sanitization and Deterministic Verification Against Adversarial Intents

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a preregistered paired confirmatory experiment (study_id cand-2026-03) that measured whether template-based input sanitization combined with deterministic verifier-gating (and at-most-one automated repair) changes the rate at which a deterministic validator accepts LLM-generated CLI configurations under adversarially mutated intents. The frozen run used the declared generator gpt-5-mini and deterministic_netops_validator_v5 within the hosted_netops_gvr_v1 adapter. Planned episodes = 300; completed episodes = 282; complete paired outcomes used in the primary analysis = 141. Observed per-pair acceptance: Baseline 135/141 (95.7447%); Guarded 141/141 (100.0%); paired absolute difference = 0.04255319148936165; exact two-sided McNemar p = 0.03125; 95% bootstrap percentile CI (10,000 resamples, seed = 1729) = [0.014184397163120567, 0.07801418439716312]. We reconcile preregistration language vs the formal estimand, document per-condition pipeline semantics with explicit archived-artifact references, report the protocol deviation (unset runtime sampling temperature), and discuss operational interpretation and limitations constrained by synthetic scope and single-model/validator evaluation. All principal numeric claims are supported by archived execution and analysis artifacts referenced in Methods and Results.**

I. Introduction

Generative LLMs are increasingly proposed to translate high-level operator intent into device CLI, promising automation gains for NetOps. However, operational risks remain when generated outputs violate sequencing constraints, CLI grammar, or safety invariants; adversarially crafted inputs can exacerbate these risks. Prior prototype studies show that coupling LLM generation with deterministic verification and targeted repair can reduce observable errors, but existing evaluations often rely on token- or reference-match metrics that do not directly measure deployability or acceptance by a deterministic validator [3,8,9].

This work asks a narrowly scoped, operational question amenable to preregistered inference: How effective are template-based input sanitization and deterministic verifier-gating (with at-most-one automated repair) at changing the rate at which a deterministic validator accepts LLM-generated CLI configurations when inputs are adversarially mutated? The study cand-2026-03 preregistered a single formal estimand (paired_success_rate_difference_guarded_minus_baseline) and a confirmatory paired test. Crucially, because the registered generation semantics were shared_initial_candidate, the paired comparison isolates downstream guarded-pipeline effects (sanitization, gating, permitted repair, and final validation) applied to a single shared LLM draft per intent rather than differences in initial generation.

Contributions. (1) A preregistered, artifact-backed paired experiment executed end-to-end and reconciled against the archived preregistration; (2) a precise, artifact-grounded description of per-condition pipeline semantics that demonstrates shared_initial_candidate reuse; (3) quantitative confirmatory results showing a small but statistically detectable absolute change in deterministic-validator acceptance under the guarded pipeline on a synthetic adversarial-intent bank; and (4) an explicit reconciliation of preregistration natural-language hypothesis wording with the formal estimand and a disclosure of a protocol deviation (unset sampling temperature) together with reproducibility guidance.

II. Related Work

Deterministic network-verification tools are a practical foundation for validator-gated NetOps pipelines because they check forwarding and configuration invariants at the configuration or header-space level and can run fast enough for interactive uses. Classic header-space analyses and their derivatives provide reachability and invariant-checking semantics over packet headers and device-wide rule sets, while streaming/online tools (e.g., VeriFlow/NetPlumber) focus on real-time policy checking as configurations change [W1882012874,W158224344,W2122695394]. These verifiers are valuable for gating because they give binary deployability decisions (accept/reject) against explicit formal properties; their effectiveness, however, depends on the granularity and correctness of the formal specification (what is asserted as a safety invariant) and on whether the verifier models dynamic protocol-level behavior or only static configuration-level constraints.

Empirical work on LLM-assisted configuration highlights complementary limitations and opportunities. Evaluations such as NetConfEval and other recent NetOps-focused studies report that LLMs frequently produce syntactic, ordering, or semantic-sequence errors when tasked with intent-to-CLI

translation; generate–validate–repair architectures—where an LLM draft is checked by a deterministic tool and then repaired—have been demonstrated in prototype settings to reduce observable errors compared to unconstrained generation [W4399629579,W4388629193]. Survey and review articles in AIOps and telecommunications emphasize fragmented evaluation practices and call for operationally meaningful metrics (deployability, validator acceptance, semantic safety) rather than only token-level similarity or BLEU-style scores [W4411735779,W4402742293,W4383605161]. Those syntheses also note that many demonstrations remain prototype-scale and that improved benchmarks and measurement semantics are needed to compare mitigation strategies reliably.

On the security and orchestration side, prompt-injection and indirect-prompt attacks against LLM-integrated applications have been documented and motivate defenses such as input-sanitization, policy gating, and stricter tool-mediated checks in operational pipelines [W4388886073,W4407163436]. Concurrent work on tool-augmented LLM patterns (e.g., Toolformer-style approaches) shows that LLMs can be taught or orchestrated to call external checks and then use feedback to repair outputs, but these approaches depend on the fidelity of the external checks and the allowed repair scope; imperfect verifier expressiveness or permissive repair policies can convert otherwise-rejectable drafts into accepted ones without guaranteeing semantic-preservation [W4320165837,W4320165837].

Positioning. Building on these bodies of work, our preregistered experiment measures a validator-centered operational outcome (deterministic-validator acceptance) under a controlled, paired design that isolates post-generation pipeline interventions. The experiment and its artifacts explicitly tie LLM draft reuse, sanitizer application, gating decision, and any limited repair calls to archived traces so that the observed paired difference can be interpreted relative to verifier semantics and prior prototype findings rather than against token-similarity baselines.

III. Methodology

Preregistration and estimand. The study cand-2026-03 was preregistered prior to observing outcomes (preregistration_sha256: ba6ed25e84c7240fbe94f2d50b3e778f bfffd810ff2db632df8946e85f0f1373). The primary estimand is paired_success_rate_difference_guarded_minus_baseline: for each adversarial intent we define a binary outcome equal to 1 if deterministic_netops_validator_v5 would accept the final candidate for deployment and 0 otherwise. The preregistered confirmatory test is an exact two-sided McNemar on paired outcomes. Bootstrap percentile 95% CIs use 10,000 resamples with seed = 1729 (analysis/analysis_log.jsonl).

Design and task bank. The preregistered design targeted N = 150 adversarial intents (paired episodes across Baseline and Guarded). The synthetic intent templates exercise interface admin changes, MTU and VLAN updates, constrained required sequences (e.g., shutdown–change–restore patterns), and minimal protected-interface rules. The adversary model used deterministic mutation heuristics applied to templates; generation seeds and mutation taxonomy are archived in execution/task_manifest.jsonl. Scope is limited to these synthetic templates and the deterministic CLI grammar encoded in our validator; no private production data was used.

Model, validator, and orchestration. The declared generator is gpt-5-mini and the deterministic validator is deterministic_netops_validator_v5 (execution/model_configuration.json). Orchestration used the hosted_netops_gvr_v1 adapter implementing a generate–validate–repair workflow. The guarded branch applied a template-based input-sanitizer prior to gating; the sanitizer and gating policy are recorded in the task manifest payloads. The archived model_configuration.json records declared_model_version = "gpt-5-mini" and an unset temperature field (temperature = null); this is disclosed as a protocol deviation (see Disclosure and Reproducibility below).

Pipeline timeline and shared_initial_candidate semantics (artifact-grounded). Because generation semantics were registered as shared_initial_candidate, the archived execution followed this per-pair timeline (artifact references in execution/execution_manifest.json, execution/task_manifest.jsonl, and execution/responses/*-shared-initial.txt): 1) Task instantiation and adversarial mutation (execution/task_manifest.jsonl). The sanitizer output and any slot changes are recorded in the task payload when sanitization altered required fields. 2) Shared initial generation: a single LLM call produced the shared_initial_candidate for the pair and was saved to execution/responses/task-<id>-shared-initial.txt. Evidence: stage_counts.shared_initial_generation = 150 in analysis/deterministic_reconciliation.json and per-episode shared_initial_candidate files referenced by scoring artifacts (e.g., execution/scoring/task-000001-baseline.json and execution/scoring/task-000001-guarded.json). 3) Baseline scoring: deterministic_netops_validator_v5 evaluated the shared_initial_candidate; baseline scoring artifacts include validation_before and validation_after snapshots. 4) Guarded branch: template-based sanitization was applied prior to gating; the verifier-gate accepted the sanitized candidate or, if permitted by policy, allowed at-most-one automated repair LLM call. Repair-stage call traces are present for episodes that invoked repair; analysis/deterministic_reconciliation.json records repair stage_count = 6 and provider-call traces exist for those episodes. 5) Final validation: the validator re-evaluated the repaired or sanitized candidate and recorded the final guarded decision and validator trace. Representative artifacts include execution/scoring/task-000001-guarded.json which show validation_before and validation_after objects.

Missingness, exclusions and handling of technical failures. The preregistered missingness_plan specified excluding a paired unit only when both paired runs were technical failures. The archive reports completed_episodes = 282, failed_episodes = 18, and complete_pairs_included = 141 (analysis/missingness_summary.csv). Provider call accounting (analysis/deterministic_reconciliation.json) documents hosted_model_call_event_count = 156, completed = 147, failed = 9, and stage_counts: shared_initial_generation

= 150, repair = 6. All exclusions and both-run failure counts are archived and reported in Results.

IV. Results

Execution summary and primary counts. The archived execution manifest reports planned_episodes = 300, completed_episodes = 282, failed_episodes = 18 and model_calls_used = 156 (execution/execution_manifest.json). After applying the preregistered paired exclusion rule (exclude pair only when both paired runs are technical failures) the confirmatory analysis used complete_pairs = 141 (analysis/missingness_summary.csv). Condition-level totals used in the primary estimand are recorded in analysis/condition_summary.csv and reproduced here: baseline_successes = 135, baseline_failures = 6 (observed = 141), guarded_successes = 141, guarded_failures = 0 (observed = 141). The condition summary artifact path: analysis/condition_summary.csv.

Paired contingency, test, and interval. The paired contingency table archived at analysis/paired_contingency_table.csv is n11 = 135 (guarded=1, baseline=1), n10 = 6 (guarded=1, baseline=0), n01 = 0 (guarded=0, baseline=1), n00 = 0 (guarded=0, baseline=0). Using the preregistered exact two-sided McNemar test on these paired outcomes yields p = 0.03125 (confirmatory result stored in analysis/results.json and summarized in analysis/paired_contingency_table.csv). The observed paired point estimate (guarded - baseline) = 0.04255319148936165. The 95% bootstrap percentile confidence interval (method and parameters preregistered; bootstrap_resamples = 10,000; bootstrap_seed = 1729) is [0.014184397163120567, 0.07801418439716312] (see analysis/analysis_log.jsonl and analysis/paired_difference_figure.svg for bootstrap metadata and visualization). These exact numeric artifacts are archived at analysis/paired_contingency_table.csv and analysis/analysis_log.jsonl.

Discordant-case diagnostics and repair-stage association. All six discordant pairs (n10) are traceable in the archive. Provider-call accounting in analysis/deterministic_reconciliation.json records stage_counts: shared_initial_generation = 150 and repair = 6; hosted_model_call_event_count = 156 (completed = 147, failed = 9). The one-to-one correspondence between the repair-stage call count and n10 suggests the discordant conversions were associated with episodes where the guarded pipeline either applied sanitization that altered the candidate prior to gating or invoked the single allowed automated repair call. Per-episode scoring artifacts for discordant pairs contain validator before/after snapshots (validation_before and validation_after) and, when present, repair-related provider-trace references (example artifact group: execution/scoring/task-00000X-baseline.json and execution/scoring/task-00000X-guarded.json). Representative artifacts demonstrating the shared_initial_candidate reuse and validator snapshots include execution/responses/task-000001-shared-initial.txt, execution/scoring/task-000001-baseline.json, and execution/scoring/task-000001-guarded.json.

Mechanism observed in archived traces. The archived validator traces show two mechanistic pathways that yielded n10 discordances: (A) template-based sanitizer canonicalized or repaired malformed slots so the sanitized candidate passed the validator when the raw shared_initial_candidate had previously failed; or (B) the guarded policy permitted one automated repair LLM call whose output, after re-validation, satisfied the validator. The archive documents both sanitizer activity (task payloads in execution/task_manifest.jsonl) and repair-stage call traces where present (provider traces referenced from scoring artifacts). The per-episode scoring artifacts include validation_before and validation_after objects which record the validator's normalized_configuration, violation_count, and validator_id = deterministic_netops_validator_v5; these fields permit exact forensic comparison of pre- and post-guarded candidates.

Missingness sensitivity. Nine paired units were excluded because both runs were technical failures (analysis/missingness_summary.csv: both_missing_pairs = 9). Applying the conservative sensitivity that treats excluded pairs as failures yields baseline_acceptance = 135/150 = 0.9000 and guarded_acceptance = 141/150 = 0.9400 (reported in Methods and briefly summarized here as a conservative bound). The archived missingness artifact path is analysis/missingness_summary.csv; the reconciliation artifact that records completed/failed provider calls and stage counts is analysis/deterministic_reconciliation.json.

Provider-call audit and reproducibility-relevant details. Provider-call accounting (analysis/deterministic_reconciliation.json) shows hosted_model_call_event_count = 156, completed_hosted_model_call_event_count = 147, failed_hosted_model_call_event_count = 9, cache_hit_count = 0, cache_miss_count = 147, and stage_counts: shared_initial_generation = 150, repair = 6. The model configuration artifact (execution/model_configuration.json) records declared_model_version = "gpt-5-mini" and temperature = null; this unset temperature value is a protocol deviation discussed in Methods and Disclosure. The provider-call audit plus the shared_initial_generation count provide artifact-grounded evidence that the per-pair generation semantics used a single shared initial candidate (see execution/responses/*-shared-initial.txt) and that at most one repair-stage call occurred for the small subset of episodes that needed it. Paths: analysis/deterministic_reconciliation.json and execution/model_configuration.json.

Representative per-task artifact inspection. For reproducibility and reader verification we point to representative per-task artifact groups archived in execution/ (examples listed in the evidence bundle): - task-000001: execution/responses/task-000001-shared-initial.txt, execution/scoring/task-000001-baseline.json, execution/scoring/task-000001-guarded.json; - task-000002..task-000006: analogous shared-initial, baseline,

and guarded response and scoring artifacts (see execution/responses/ and execution/scoring/). These representative artifacts show identical shared_initial_candidate content referenced by both baseline and guarded scoring records along with validator validation_before/validation_after snapshots and the scorer_id = deterministic_netops_validator_v5, enabling exact verification of the reported paired outcomes.

Unexecuted preregistered items and limitations relevant to results. The preregistered benign-control (M = 50) and false-positive-blocking secondary estimand were not executed; the execution/task_manifest.jsonl and responses directories contain no benign-control artifacts. Therefore no empirical false-positive-blocking rates are reported in this run and none are claimed. All the numerical summaries above (counts, contingency, p-value, CI, provider-call audit) are directly supported by the archived artifacts cited above (analysis/paired_contingency_table.csv; analysis/condition_summary.csv; analysis/missingness_summary.csv; analysis/deterministic_reconciliation.json; execution/responses/*-shared-initial.txt; execution/scoring/*.json).

V. Discussion

Hypothesis reconciliation and interpretive boundary. The preregistration's natural-language H1 asserted that verifier-gating would reduce attacker success (i.e., fewer unsafe configurations accepted). The registered formal estimand measured paired validator acceptance-for-deployment (guarded - baseline). These constructs differ: an increase in validator acceptance can reflect successful repair of malformed but benign drafts and not necessarily increased attacker success. Our observed guarded→higher acceptance (≈4.255 percentage points) occurred because sanitizer and permitted repair recovered reparable drafts that the baseline validator rejected. Therefore the preregistered directional statement about reducing attacker success is not supported by the formal-estimand outcome; we report and interpret the archived formal estimand explicitly and caution against conflating validator acceptance with attacker-centric safety without an independent semantic-safety oracle.

Operational implications and repair–gate co-design. Within the synthetic setting, input sanitization plus bounded automated repair can reduce operator rework by converting invalid drafts into validator-accepted candidates. However, this benefit depends on the validator's semantic coverage and the scope of permitted repairs: the six discordant cases show repairs enabling acceptance, which is beneficial if repairs preserve intended semantics but risky if repairs mask adversarial manipulations. This highlights an operational co-design requirement: gating policies, repair scope, and validator expressiveness must be jointly specified to preserve safety in deployment.

Threats to validity. Internal validity: 18 technical failures reduced the effective sample and triggered the preregistered exclusion rule (both-run failures = 9). Construct validity: deterministic_netops_validator_v5 encodes CLI grammar, required sequences, and protected-interface invariants but does not capture all dynamic protocol-level semantics; acceptance-for-deployment is therefore a partial proxy for operational safety. External validity: a single generator (gpt-5-mini), single validator, and synthetic intent bank limit generalization across vendors, CLI dialects, and production traffic. Conclusion validity: the absolute effect is small against a high baseline (ceiling effect); statistical detectability does not imply broad practical impact.

Reproducibility guidance and protocol deviation. The archived execution/model_configuration.json recorded temperature = null (unset). We disclose this as a protocol deviation because provider-side defaults can affect sampling. To preserve paired comparability despite the unset parameter, the pipeline employed shared_initial_candidate semantics (single generation per pair) and provider-call accounting (analysis/deterministic_reconciliation.json) documents shared_initial_generation = 150 and repair = 6, confirming the actual call pattern. For deterministic reproduction we recommend explicitly setting temperature = 0 and re-executing the archived execution/task_manifest.jsonl with identical provider semantics; artifact locations referenced above (execution/* and analysis/*) permit exact artifact-based verification of outcomes.

Recommendations. Based on archived evidence: (1) execute the preregistered benign-control to measure false-positive blocking; (2) preregister an attacker-centric estimand that couples validator acceptance with an independent semantic-safety oracle or reachability tests; (3) diversify generators and validators across vendors and CLI dialects; and (4) fix sampling settings (temperature = 0) for deterministic reproduction. Each recommendation follows directly from the observed artifacts and provider-call accounting.

VI. Limitations

- Synthetic task bank and intent templates — not real production traffic or operator logs; results may not generalize to field datasets.
- Single-model evaluation (gpt-5-mini) and single validator implementation limit external validity across vendors, protocols, and model families.
- Validator coverage constrained to encoded safety properties (CLI grammar, required sequences, protected-interface invariants); some protocol-level or dynamic runtime semantics are not represented.
- Technical failures (18/300 episodes) reduced effective sample size; paired exclusion (both-run failures = 9) was preregistered and applied.
- Adversary model limited to mutations of synthetic intents; prompt-injection in retrieval-augmented or multi-source pipelines was not exhaustively evaluated.
- A preregistration natural-language hypothesis phrasing differed from the registered formal estimand; results are reported and interpreted with respect to the archived formal estimand.
- The preregistered benign-control (M = 50) and false-positive-blocking secondary estimand were not executed;

therefore no empirical false-positive-blocking claims are made.

VII. Conclusion

A preregistered, artifact-backed paired experiment on a synthetic adversarial intent bank found that template-based input sanitization combined with deterministic verifier-gating and at-most-one automated repair produced a small but statistically detectable absolute increase in deterministic-validator-accepted LLM-generated configurations under the archived formal estimand (paired difference $\approx$ 0.04255; p = 0.03125; 95% CI [0.01418, 0.07801]). Diagnostics show the six discordant pairs were associated with permitted repair calls that enabled acceptance. Importantly, this formal-estimand improvement does not equate to measured reduction in attacker success under attacker-centric definitions. The run is limited by synthetic scope, a single model/validator, and a protocol deviation (unset runtime temperature). Future work should execute the preregistered benign-control, measure attacker-centric estimands with an independent semantic-safety oracle, diversify models/validators, and set explicit sampling parameters for reproducible deterministic runs. All principal numbers and reconciliation claims above are supported by archived execution and analysis artifacts referenced in Methods and Results (e.g., execution/task_manifest.jsonl; execution/responses/task-000001-shared-initial.txt; execution/scoring/task-000001-baseline.json; execution/scoring/task-000001-guarded.json; analysis/paired_contingency_table.csv; analysis/condition_summary.csv).

Disclosure Statement

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256: 1872df1e1805d2d9 6940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role and autonomy boundary: a human Research Director selected the topic family (Generative AI / LLMs for NetOps) and constrained the initial master prompt prior to the autonomy lock; all literature review, preregistration, study selection, code generation, experiment execution, analysis, peer-review cycles, manuscript drafting, and revision reported here were performed autonomously after the lock. Declared models/tools and orchestration: primary generator = gpt-5-mini (execution/model_configuration.json), deterministic validator = deterministic_netops_validator_v5, task generator = deterministic_netops_task_generator_v5, orchestration adapter family = hosted_netops_gvr_v1. Preregistration: study cand-2026-03 was preregistered and archived prior to observing outcomes (preregistration_sha256: ba6ed25e 84c7240fbe94f2d50b3e778fbfffd810ff2db632df8946e8 5f0f1373). Protocol deviation: the archived run recorded an unset runtime sampling temperature (temperature = null in execution/model_configuration.json); this is disclosed as a deviation because provider-side defaults may affect sampling determinism. To preserve paired comparability the pipeline used shared_initial_candidate semantics (single shared generation per pair) and provider-call accounting documented in analysis/deterministic_reconciliation.json confirms the actual call pattern (shared_initial_generation = 150; repair = 6). Key archived artifact references (examples): execution/task_manifest.jsonl; execution/responses/task-000001-shared-initial.txt; execution/scoring/task-000001-baseline.json; execution/scoring/task-000001-guarded.json; analysis/deterministic_reconciliation.json; analysis/paired_contingency_table.csv; analysis/condition_summary.csv; analysis/analysis_log.jsonl; execution/model_configuration.json; execution/execution_manifest.json. No human manually edited or corrected the manuscript technical content after the autonomy lock.

References

[1] http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.300.8659
[2] https://doi.org/10.1145/2342441.2342452
[3] https://doi.org/10.1145/3626111.3628194
[4] https://doi.org/10.1145/3605764.3623985
[5] http://arxiv.org/abs/2302.04761
[6] https://doi.org/10.1145/3656296
[7] Hao Zhou, Chengming Hu, Ye Yuan, Yufei Cui, Yili Jin, Can Chen, Haolun Wu, Dun Yuan, Li Jiang, Di Wu, Xue Liu, Jianzhong Zhang, Xianbin Wang, Jiangchuan Liu, "Large Language Model (LLM) for Telecommunications: A Comprehensive Survey on Principles, Key Techniques, and Opportunities", 2024, 10.1109/comst.2024.3465447
[8] Lingzhe Zhang, Tong Jia, Mengxi Jia, Yifan Wu, Aiwei Liu, Yong Yang, Zhonghai Wu, Xuming Hu, Philip S. Yu, Ying Li, "A Survey of AIOps in the Era of Large Language Models", 2025, 10.1145/3746635
[9] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194
[10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, Mario Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection", 2023, 10.1145/3605764.3623985
[11] Yupeng Chang, Xu Wang, Jindong Wang, Yuan-Hsuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, Xing Xie, "A Survey on Evaluation of Large Language Models", 2023, 10.48550/arxiv.2307.03109
[12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Răileanu, María Lomelí, Luke Zettlemoyer, Nicola Cancedda, Thomas Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools", 2023, 10.48550/arxiv.2302.04761
[13] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296
[14] Peyman Kazemian, George Varghese, Nick McKeown, "Header space analysis: static checking for networks", 2012